

Neural Network

Weihaio Wang
Department of Applied Mathematics and Statistics
Stony Brook University



What is NN?

- Neural networks are a core **AI-based** method. They are a subset of machine learning that mimics the structure and function of the human brain, enabling computers to learn from data, recognize patterns, and make decisions.
- **Definition:** A computational model inspired by the human brain.
- **Purpose:** Designed to recognize patterns and solve complex problems. (Classification or Regression)



Neuron Model

- Each neuron computes a weighted sum of its inputs, adds a bias, and then passes the result through an activation function:

$$y = f \left(\sum_{i=1}^n w_i x_i + b \right)$$

- By adjusting the weights and biases during training, the network learns to perform tasks like classification or regression.

How Neural Networks Work

- **Layered Structure:**

Neural networks are organized into layers:

- **Input Layer:** Receives raw data.
- **Hidden Layers:** Process data via weighted transformations.
- **Output Layer:** Produces the final prediction.



Forward Propagation

- For each layer l , the computation is:

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$$

$$a^{(l)} = f(z^{(l)})$$

- W : Weight matrix of layer l
- a : Activations from the previous layer
- b : Bias vector
- f : Activation function



Backproagation

- After the forward pass, the network calculates the error at the output and propagates it backward to **adjust the parameters using the chain rule.**



The NN algorithm

- **Step 1: Forward Propagation**

- Compute activations through each layer $a^{(l)} = f(W^{(l)}a^{(l-1)} + b^{(l)})$

- **Step 2: Loss Calculation**

- Calculate the loss L by comparing the network's output y_{hat} to the true output y (MSE or Cross-Entropy)
- $L = \text{loss}(y, y_{\text{hat}})$

- **Step 3: Backpropagation**

- Compute gradients of the loss with respect to each weight using the chain rule

$$\frac{\partial L}{\partial W^{(l)}}$$

- **Step 4: Weight Update**

- Update the weights using gradient descent

$$W^{(l)} = W^{(l)} - \alpha \frac{\partial L}{\partial W^{(l)}}$$



Parameters in NN

- **Trainable Parameters:**

- **Weights (W):** Determine the strength of connections between neurons.
- **Biases (b):** Provide an offset for the weighted sum.

- **Hyperparameters:**

- **Learning Rate (α):** Controls the size of weight updates.
- **Epochs:** Number of complete passes through the training dataset.
- **Batch Size:** Number of samples processed before updating weights.
- **Architecture:** Number of layers and neurons per layer.

- **Weight Update Formula:**

- This iterative update minimizes the loss function over time.

$$W_{ij} = W_{ij} - \alpha \frac{\partial L}{\partial W_{ij}}$$



Hidden Layers

- **Functionality:**

Hidden layers allow the network to learn **complex, non-linear relationships** by transforming the input data into increasingly abstract representations.

- **Mathematical Representation:**

$$a^{(l)} = f \left(W^{(l)} a^{(l-1)} + b^{(l)} \right)$$

Each hidden layer applies a non-linear activation function f to capture complex patterns.



Design Considerations

- **Depth vs. Width:** **Increasing the number of layers (depth) or neurons per layer (width)** can enhance the network's learning capacity, but may also lead to **overfitting** if not managed properly.
- **Overfitting** occurs when a neural network model learns not only the underlying patterns in the training data but also the noise and random fluctuations. As a result, while the model performs very well on the training data, its performance degrades significantly on unseen or test data.

Loss Functions: CE and SSE

- **Cross-Entropy Loss (CE):**
 - **Usage:** Primarily used for **classification** problems.
 - **Formula for Multi-Class Classification:**

$$L_{CE} = - \sum_{i=1}^N y_i \log(\hat{y}_i)$$

- **Explanation:**

Measures the difference between the true probability distribution y and the predicted probability distribution y_{hat} .

Loss Functions: CE and SSE

- **Sum of Squared Errors (SSE):**
 - **Usage:** often used for **regression** tasks.
 - **Formula:**

$$L_{SSE} = \frac{1}{2} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

- **Explanation:**

Quantifies the squared difference between the predicted outputs and the actual values.



Activation Functions

- **Purpose:**

Introduce non-linearity, allowing neural networks to learn complex patterns.

- **Common Activation Functions:**

- **Sigmoid Function:**

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- Outputs values in the range (0, 1), ideal for binary classification.

- **Tanh Function:**

- Outputs values in the range (-1, 1), providing centered data. $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$

- **ReLU (Rectified Linear Unit):**

- Efficient for computation; helps mitigate the vanishing gradient problem.

$$f(x) = \max(0, x)$$



Summary

- **Neural Networks:** Computational models inspired by the brain that learn from data.
- **Core Process:** Involves forward propagation, loss computation, backpropagation, and parameter updates.
- **Components:** Weights, biases, hidden layers, activation functions, and loss functions (CE vs. SSE).
- Understanding these formulas and the flow of information through the network is crucial for designing, training, and optimizing neural networks.